



응용프로그래밍 팀 프로젝트

KOREA TECH TERM PROJECT

목차

CONTENTS

01

프로젝트 설명

PROJECT
SUMMARY

02

동작 원리

PROJECT
FUNCTION

03

동작 구현

TECHNOLOGY
STACK

04

플레이 영상

MEMBERS

프로젝트 설명

PROJECT SUMMARY

프로젝트 명: 뱀게임(snake game)

PROJECT NAME

뱀게임 규칙

RULE

- 플레이어가 키보드의 방향키를 사용하여 뱀의 위치를 조종할 수 있습니다.
- 먹이를 15개 먹으면 게임이 클리어 되고 종료됩니다.
- 먹이를 먹으면 뱀의 몸통이 증가하고 점수를 1점 얻습니다.
- 벽에 부딪히거나 자신의 몸통에 부딪히면 게임이 종료됩니다.

특징

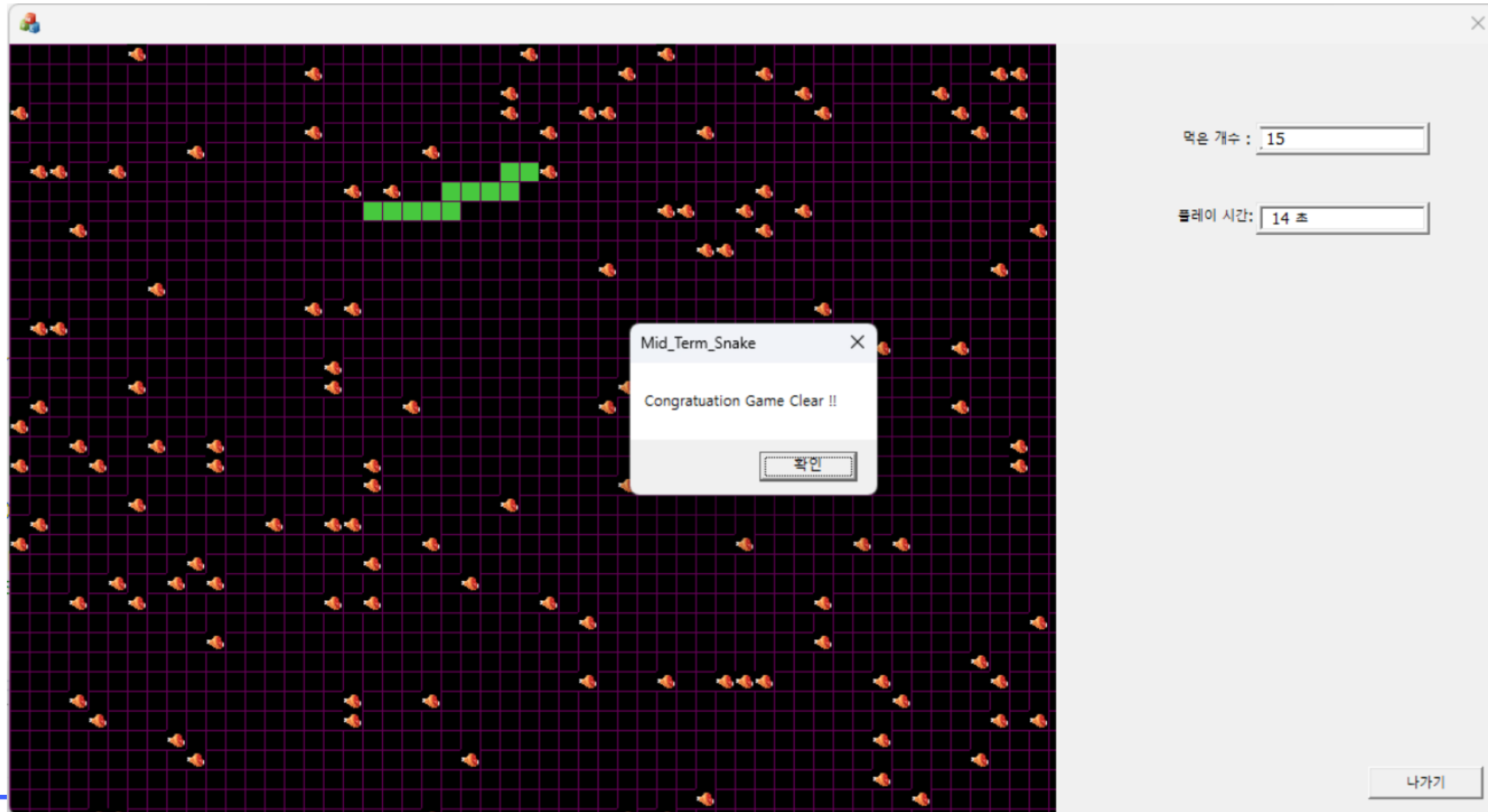
FEATURE

- 검은색 격자 화면이 바탕이고 뱀의 먹이는 고기 이미지를 bitmap 형식으로 구현하였습니다.



- 뱀의 몸통은 RGB값을 이용하여 초록색으로 구현하였습니다.
- 타이머를 사용하여 뱀의 몸통 및 움직임을 구현하였고 플레이어가 얼마나 게임을 플레이 하였는지 알 수 있도록 플레이 타임도 구현하였습니다.

뱀게임 화면 구성



- 뱀이 벽이나 자신의 몸에 부딪히면 게임이 종료되고 "GAME OVER" 라는 메시지가 나오면서 게임이 종료됩니다
- 플레이어가 뱀을 조종하여 먹이를 먹으면 오른쪽 화면에 점수가 표시됩니다. (먹은 개수:)
- 플레이어가 얼마나 플레이 하였는지 시간을 나타낼 수 있습니다. (플레이 시간:)
- '확인' 버튼이나 '나가기' 버튼으로 게임이 종료된후 게임을 나갈 수 있습니다.

동작 원리

TECHNOLOGY STACK

함수 MAP()

```
void CMidTermSnakeDlg::MAP()
```

```
{  
    m_draw_dc.SelectStockObject(DC_BRUSH);  
    m_draw_dc.SelectStockObject(DC_PEN);  
    m_draw_dc.SetDCPenColor(RGB(102, 0, 88)); // 격자색 설정  
  
    for (int y = 0; y < 60; y++) // 화면 크기에 따른 격자 생성  
    {  
        for (int x = 0; x < 80; x++)  
        {  
            if (m_count_map[y][x] > 0) //  
            {  
                m_count_map[y][x]--; //  
  
                if (m_count_map[y][x] == 0) m_table[y][x] = 0; // 카운트가 0이되면 검은색이 되게  
            }  
  
            m_draw_dc.SetDCBrushColor(m_table[y][x]);  
            m_draw_dc.Rectangle(x * 15, y * 15, x * 15 + 16, y * 15 + 16); // 격자 크기 조정  
        }  
    }  
  
    DrawFood(); // 먹이 그리는 함수 호출하여 화면에 먹이 출력  
}
```

```
COLORREF m_table[60][80] = { 0 }; // 데이터 저장 가로80 세로60 + 색상변경까지  
int m_count_map[60][80] = { 0 }; // 뱀이 있었던 자리를 카운트로 값을 받아온다
```

- 뱀의 잔상을 MAP()함수 구현하였습니다.

m_count_map에 카운트 값을 저장합니다.

eat_count +2 값이 저장되며 타이머에 의해서 한칸씩 이동할때 --로 값을 줄여뱀이 이동할때 잔상을 남겨 길어지는것을 표현하였습니다.

동작 원리

TECHNOLOGY STACK

함수 DrawFood()

```
srand((unsigned int)time(NULL)); //랜덤 함수로 먹이 구현
int x, y;
for (int i = 0; i < 150; i++)
{
    x = rand() % 53;
    y = rand() % 40;
    if (m_table[y][x] == 0) m_table[y][x] = RGB( 255,255,255); // 중복된 값 처리
    else i--;
}
```

- 먹이 생성은 srand 랜덤 함수를 사용하여서 격자 크기에 맞게 150개를 랜덤으로 먹이를 생성하도록 하였습니다.
- RGB값을 이용하여 255,255,255 즉 흰색 객체를 먹이로 설정하였습니다.

```
// 비트맵 이미지 선택
CBitmap* pOldBitmap = memDC.SelectObject(&m_bmpFood);
BITMAP bmpInfo;
m_bmpFood.GetBitmap(&bmpInfo);

// 격자 크기 안에서 먹이(파란색)를 생성하고 이 위치에 비트맵 이미지로 먹이 그리기
for (int y = 0; y < 60; y++)
{
    for (int x = 0; x < 80; x++)
    {
        if (m_table[y][x] == RGB(255, 255, 255)) // 테이블에서 흰색(먹이)인 경우 설정
        {
            m_draw_dc.TransparentBlt(x * 15, y * 15, 15, 15, &memDC, 0, 0, bmpInfo.bmWidth, bmpInfo.bmHeight, RGB(255, 255, 255)); // 이미지를 불러온다.
        }
    }
}

memDC.SelectObject(pOldBitmap);
```

- 먹이에 BITMAP 이미지를 RGB흰색(먹이)에 삽입합니다.
- TransparentBlt() 를 사용해서 각 격자 크기와 이미지의 너비 높이에 맞게 이미지를 불러옵니다.

동작 원리

TECHNOLOGY STACK

함수 게임종료 및 클리어

```
void CMidTermSnakeDlg::StopGame()
{
    KillTimer(1);
    KillTimer(2);
    MessageBox(_T("GAME OVER"));
}

void CMidTermSnakeDlg::ClearGame()
{
    KillTimer(1);
    KillTimer(2);
    MessageBox(_T("Congratuation Game Clear !!"));
}
```

- 게임이 종료될때 타이머를 종료시키기 위해서 StopGame() 과 ClearGame()함수를 구현하였습니다.

동작 원리

TECHNOLOGY STACK

뱀의 몸통 구현 (Timer=1)

```
if (nIDEvent == 1)
{
    if (m_direction == 0)    m_pos.x--; // left 뱀의 위치를 구현
    else if (m_direction == 1) m_pos.y--; // up
    else if (m_direction == 2) m_pos.x++; // right
    else if (m_direction == 3) m_pos.y++; // down

    if (m_pos.x >= 0 && m_pos.x <= 79 && m_pos.y >= 0 && m_pos.y <= 59) // 울타리 범위 제한
    {
        if (m_table[m_pos.y][m_pos.x] == RGB(255, 255, 255)) //뱀이 먹이를 먹은 상황
        {
            m_eat_count++; // 먹은것으로 간주
            SetDlgItemInt(IDC_EAT_COUNT_EDIT, m_eat_count);

            if (m_eat_count == 15) // 먹이 15개를 먹었을 때 게임 클리어
            {
                ClearGame();
                return;
            }
        }
    }
}
```

- 가장 중요한 부분인 뱀의 몸통을 구현하는 타이머 부분입니다.
- 뱀을 타이머에 m_direction 값에 키보드로 값을 입력 받아 x, y값을 조정하여 움직일 수 있습니다.
- m_eat_count는 RGB(255,255,255) 일때 값이 증가하게 되며, m_eat_count 에 값이 15가 되면 게임이 클리어 되고 종료됩니다.

동작 원리

TECHNOLOGY STACK

뱀의 몸통 구현 (Timer=1)

```
else if (m_table[m_pos.y][m_pos.x] != 0) // 경우 자신의 꼬리에 닿고 게임 오버
{
    StopGame();
    return;
}
if (m_pos.x < 0 || m_pos.x >= 53 || m_pos.y < 1 || m_pos.y >= 40) // 뱀이 경기장 울타리에 부딪혔을때 게임오버
{
    StopGame();
    return;
}

m_table[m_pos.y][m_pos.x] = RGB(71, 200, 62); // 이동한 위치에서 뱀(초록색) 을 다시 나타냄
m_count_map[m_pos.y][m_pos.x] = m_eat_count + 2;

MAP();

CClientDC dc(this);
m_draw_image.Draw(dc, 0, 0);
}
else
{
    KillTimer(1); //타이머 멈춤
    MessageBox(_T("GAME OVER"));
}
```

- 게임이 종료 되는 조건을 구현해 두었습니다.
- 조건 1. m_table[m_pos.y][m_pos.x] !=0 으로 검은색배경이 아닌 다른 색을 만나면 즉, 자신의 몸통에 부딪히면 종료됩니다.
- 조건 2. if 문을 사용해서 경기장 크기 , 격자 크기인 800*600 밖에 나가면 게임이 종료되게 하였습니다.

동작 원리

TECHNOLOGY STACK

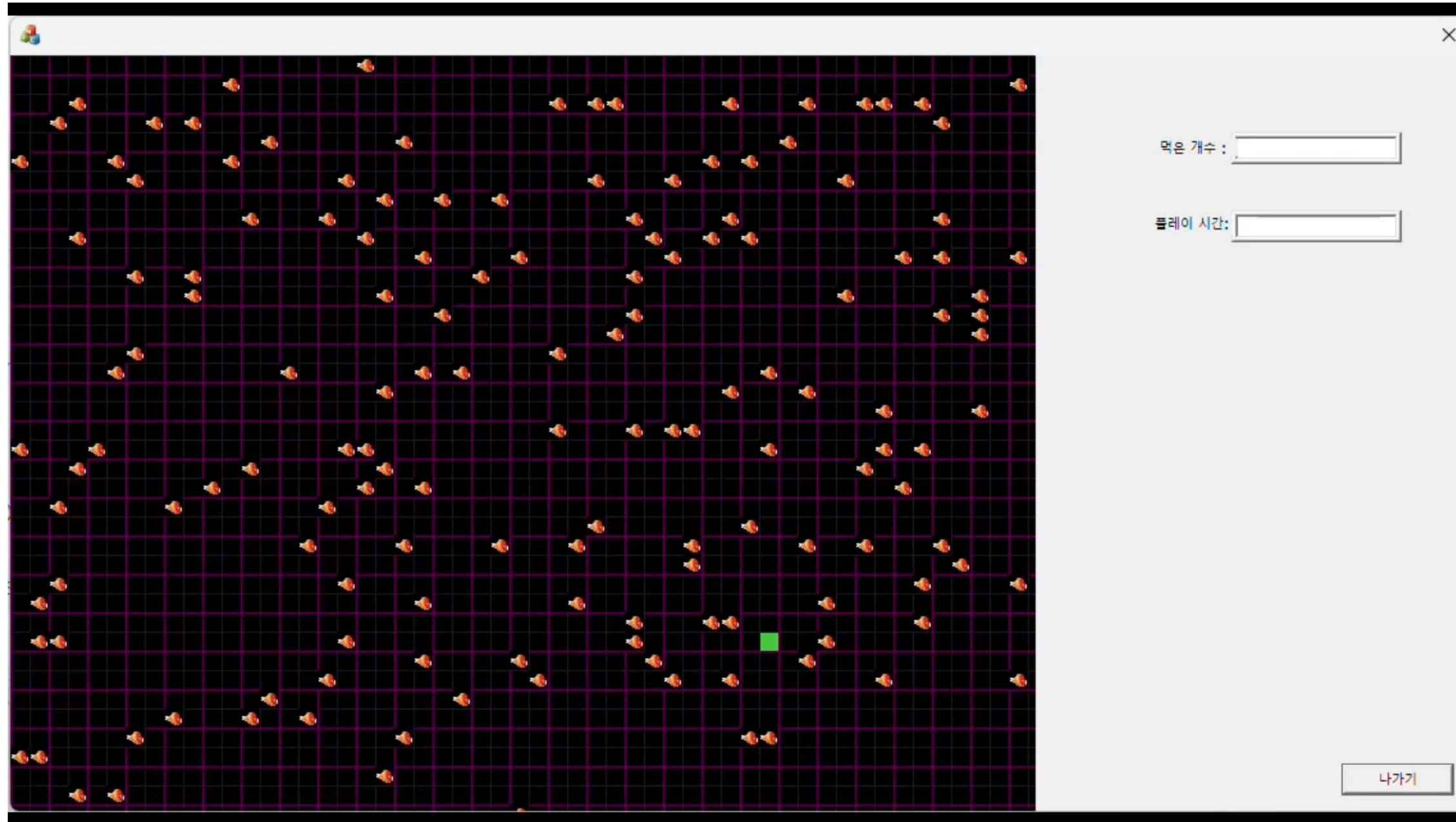
플레이 타임 구현 (Timer = 2)

```
else if (nIDEvent == 2) // 뱀 구현과 따로 해줌
{
    // 플레이 타임 업데이트
    m_playTime++;
    CString strPlayTime;
    strPlayTime.Format(_T(" %d 초"), m_playTime);
    SetDlgItemText(IDC_playTime, strPlayTime);
}
CDialogEx::OnTimer(nIDEvent);
```

- 타이머를 사용하여 플레이어가 얼마나 게임을 플레이 했는지 나타냅니다.
- 벽에 부딪히거나 뱀이 자신의 몸통에 부딪혔을때 게임이 종료되므로 타이머도 같이 꺼지게 설계 하였습니다.

간단한 플레이 영상

MEMBERS



- 15~20초 정도 되는 플레이 영상을 첨부 하였습니다.
- 벽에 부딪히거나 뱀이 자신의 몸통에 부딪혔을때 게임이 종료되므로 타이머도 같이 꺼지게 설계 하였습니다.



감사합니다.

응용프로그래밍 텀프로젝트



2022161137 전주영